JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.file

Interface Watchable

All Known Subinterfaces:

Path

`public interface Watchable`

An object that may be registered with a watch service so that it can be *watched* for changes and events.

This interface defines the `register` method to register the object with a `WatchService` returning a `WatchKey` to represent the registration. An object may be registered with more than one watch service. Registration with a watch service is cancelled by invoking the key's `cancel` method.

Since:

1.7

See Also:

`Path.register(java.nio.file.WatchService, java.nio.file.WatchEvent.Kind<?>[], java.nio.file.WatchEvent.Modifier...)`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
`WatchKey`	`register(WatchService watcher, WatchEvent.Kind<?>... events)`	Registers an object with a watch service.

WatchKey	**register**(**WatchService** watcher, **WatchEvent.Kind**<?>[] events, **WatchEvent.Modifier**... modifiers)	Registers an object with a watch service.

Method Detail

register

```
WatchKey register(WatchService watcher,
                  WatchEvent.Kind<?>[] events,
                  WatchEvent.Modifier... modifiers)
           throws IOException
```

Registers an object with a watch service.

If the file system object identified by this object is currently registered with the watch service then the watch key, representing that registration, is returned after changing the event set or modifiers to those specified by the `events` and `modifiers` parameters. Changing the event set does not cause pending events for the object to be discarded. Objects are automatically registered for the `OVERFLOW` event. This event is not required to be present in the array of events.

Otherwise the file system object has not yet been registered with the given watch service, so it is registered and the resulting new key is returned.

Implementations of this interface should specify the events they support.

Parameters:

`watcher` - the watch service to which this object is to be registered

`events` - the events for which this object should be registered

`modifiers` - the modifiers, if any, that modify how the object is registered

Returns:

a key representing the registration of this object with the given watch service

Throws:

`UnsupportedOperationException` - if unsupported events or modifiers are specified

`IllegalArgumentException` - if an invalid of combination of events are modifiers are specified

`ClosedWatchServiceException` - if the watch service is closed

`IOException` - if an I/O error occurs

SecurityException - if a security manager is installed and it denies an unspecified permission required to monitor this object. Implementations of this interface should specify the permission checks.

register

```
WatchKey register(WatchService watcher,
                  WatchEvent.Kind<?>... events)
           throws IOException
```

Registers an object with a watch service.

An invocation of this method behaves in exactly the same way as the invocation

```
watchable.register(watcher, events, new WatchEvent.Modifier[0]);
```

Parameters:

watcher - the watch service to which this object is to be registered

events - the events for which this object should be registered

Returns:

a key representing the registration of this object with the given watch service

Throws:

UnsupportedOperationException - if unsupported events are specified

IllegalArgumentException - if an invalid of combination of events are specified

ClosedWatchServiceException - if the watch service is closed

IOException - if an I/O error occurs

SecurityException - if a security manager is installed and it denies an unspecified permission required to monitor this object.

```
Implementations of this interface should specify the permission
checks.
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.